



Chapter 7 – Design and Implementation

Topics covered



- ✧ Design and implementation
- ✧ Object-oriented design using the UML
- ✧ Design patterns
- ✧ Implementation issues and Software Reuse

Design and implementation



- ✧ Software design and implementation is the stage in the software engineering process at which an **executable software system is developed**.
- ✧ Software design and implementation **activities may be interleaved**.
 - Software design is a **creative activity** in which you **identify software components and their relationships, based on a customer's requirements**.
- ✧ **Implementation** - the process of realizing the design as a program.

Build or buy



- ✧ In a wide range of domains, it is now possible to **buy off-the-shelf systems** that can be **adapted and tailored** to the users' requirements.
 - E.g, a **medical records system**, you can buy a package
 - It can be **cheaper** and **faster** to use.

- ✧ There is a **concerned with how to use the configuration features** of such system.



Object-oriented design using the UML

An object-oriented design process



✧ Common activities in object-oriented design processes:

- Define the **context and modes of use** of the system;
- Design the **system architecture**;
- Identify the **principal system objects**;
- Develop **design models**;
- Specify **object interfaces**.

✧ Process illustrated here using a **design for a wilderness weather station.**

System context and interactions



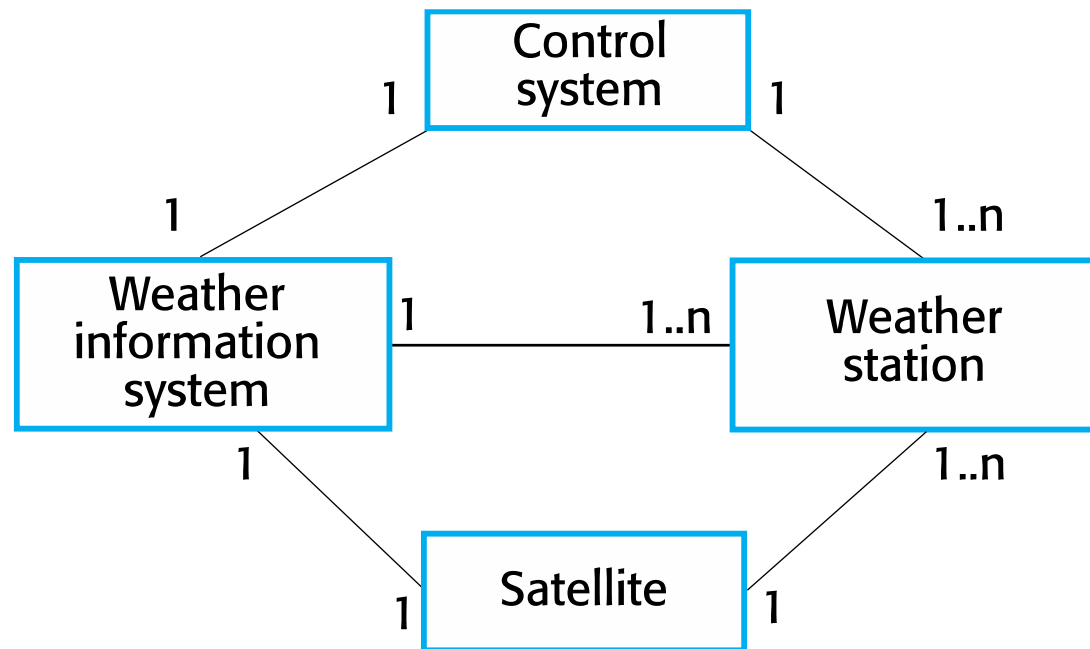
- ✧ Understanding the **relationships between**
- ✧ The software that is being designed and
- ✧ Its external environment
- ✧ Setting the system boundaries.

Context and interaction models

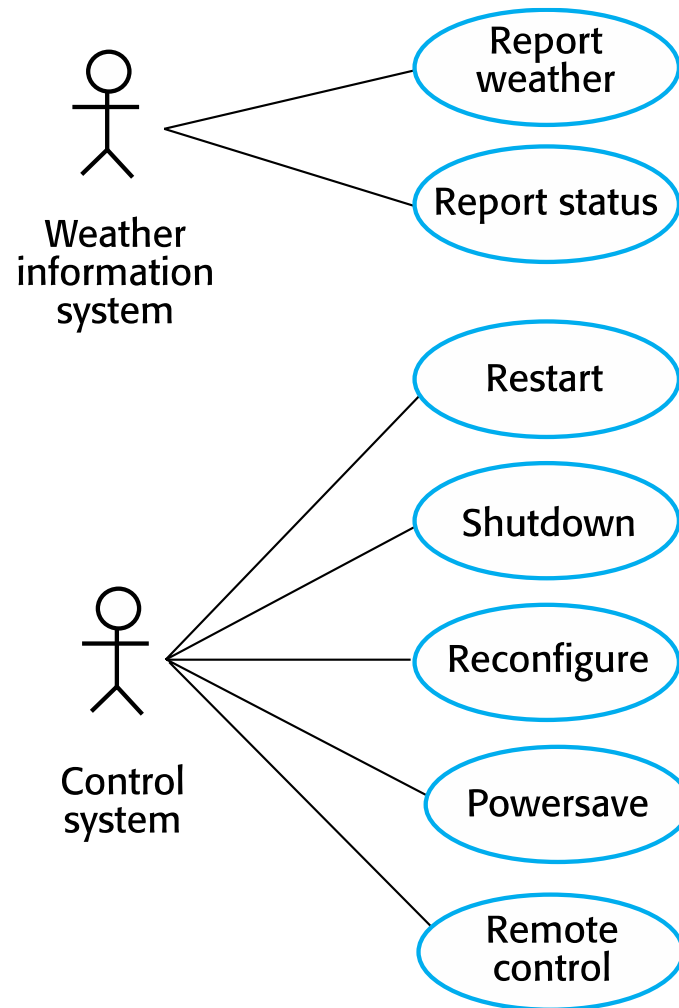


- ✧ **A system context model** is a **structural model** that demonstrates the other systems in the environment.
- ✧ **An interaction model** is a **dynamic model** - system interaction with its environment.

System context for the weather station



Weather station use cases



Use case description—Report weather



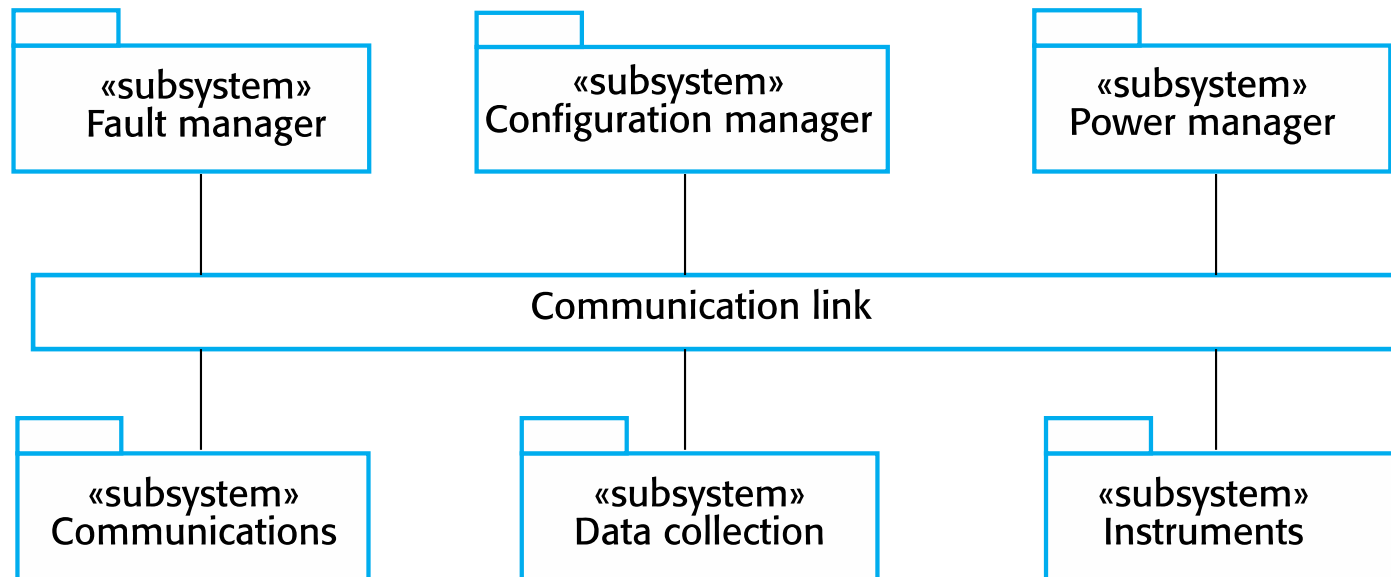
System	Weather station
Use case	Report weather
Actors	Weather information system, Weather station
Description	The weather station sends a summary of the weather data that has been collected from the instruments in the collection period to the weather information system. The data sent are the maximum, minimum, and average ground and air temperatures; the maximum, minimum, and average air pressures; the maximum, minimum, and average wind speeds; the total rainfall; and the wind direction as sampled at five-minute intervals.
Stimulus	The weather information system establishes a satellite communication link with the weather station and requests transmission of the data.
Response	The summarized data is sent to the weather information system.
Comments	Weather stations are usually asked to report once per hour but this frequency may differ from one station to another and may be modified in the future.

Architectural design

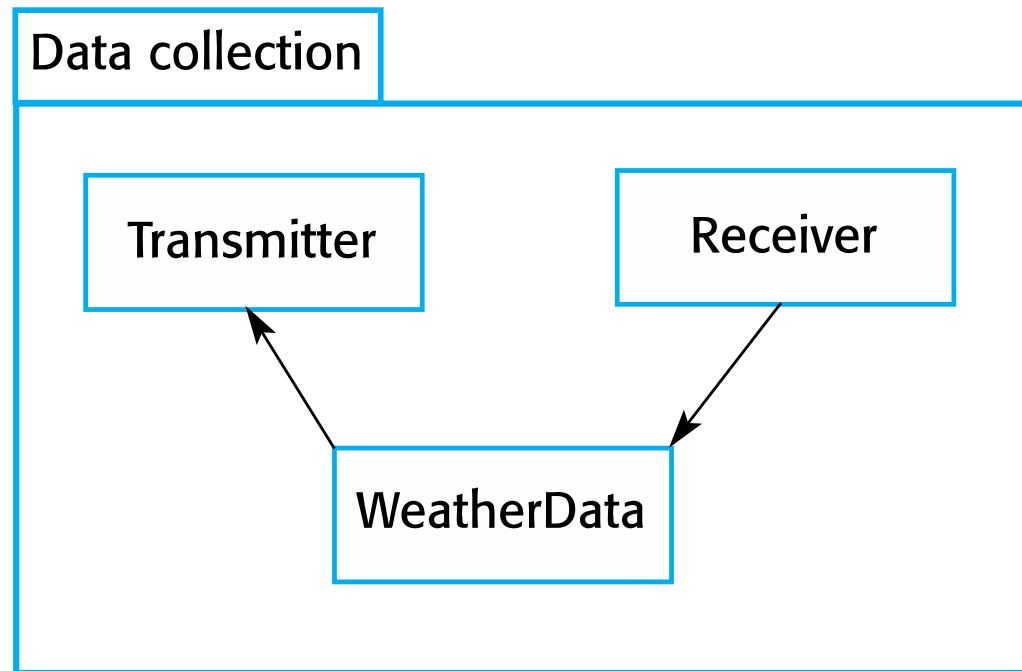


- ✧ Use the established information for **designing the system architecture**.
 - **Identify the major components** that make up the system and their interactions
 - **Organize the components** using an architectural pattern
 - such as a **layered** or **client-server model**.

High-level architecture of the weather station



Architecture of data collection system



Object class identification



- ✧ There is **no 'magic formula'** for object identification.
 - It relies on the **skill, experience, and domain knowledge** of system designers.
- ✧ Object identification is an **iterative process**.
 - You are unlikely to get it right the first time.

Approaches to identification



- ✧ Use a **grammatical approach** - natural language description of the system.
- ✧ **Tangible things** in the application domain are identified
- ✧ Use a **behavioural approach** - identify objects based on what participates in the behaviour.
- ✧ Use a **scenario-based analysis**. - identify objects, attributes, and methods in each scenario.

Weather station object classes



✧ Object class identification in the weather station system may be based on the **tangible hardware** and **data** in the system:

- **Ground thermometer, Anemometer, Barometer**
 - Application domain objects that are 'hardware' objects related to the instruments in the system.
- **Weather station**
 - The basic interface of the weather station to its environment..
- **Weather data**
 - Encapsulates the summarized data from the instruments.

Weather station object classes



WeatherStation
identifier
reportWeather () reportStatus () powerSave (instruments) remoteControl (commands) reconfigure (commands) restart (instruments) shutdown (instruments)

WeatherData
airTemperatures groundTemperatures windSpeeds windDirections pressures rainfall
collect () summarize ()

Ground thermometer
gt_Ident temperature
get () test ()

Anemometer
an_Ident windSpeed windDirection
get () test ()

Barometer
bar_Ident pressure height
get () test ()

Design models



✧ Design models show the **objects and object classes and relationships between these entities**.

✧ **Two kinds of design model:**

- **Structural models** describe the **static structure** of the system in terms of object classes and relationships.
- **Dynamic models** describe **the dynamic interactions between objects**.

Examples of design models



- ✧ **Sequence models** that show the sequence of object interactions.
- ✧ **State machine models** that show how individual objects change their state in response to events.
- ✧ **Subsystem models** that show logical groupings of objects into coherent subsystems.
- ✧ Other models include **use-case models**,
- ✧ **Aggregation models**,
- ✧ **Generalization models**, etc.

Sequence models

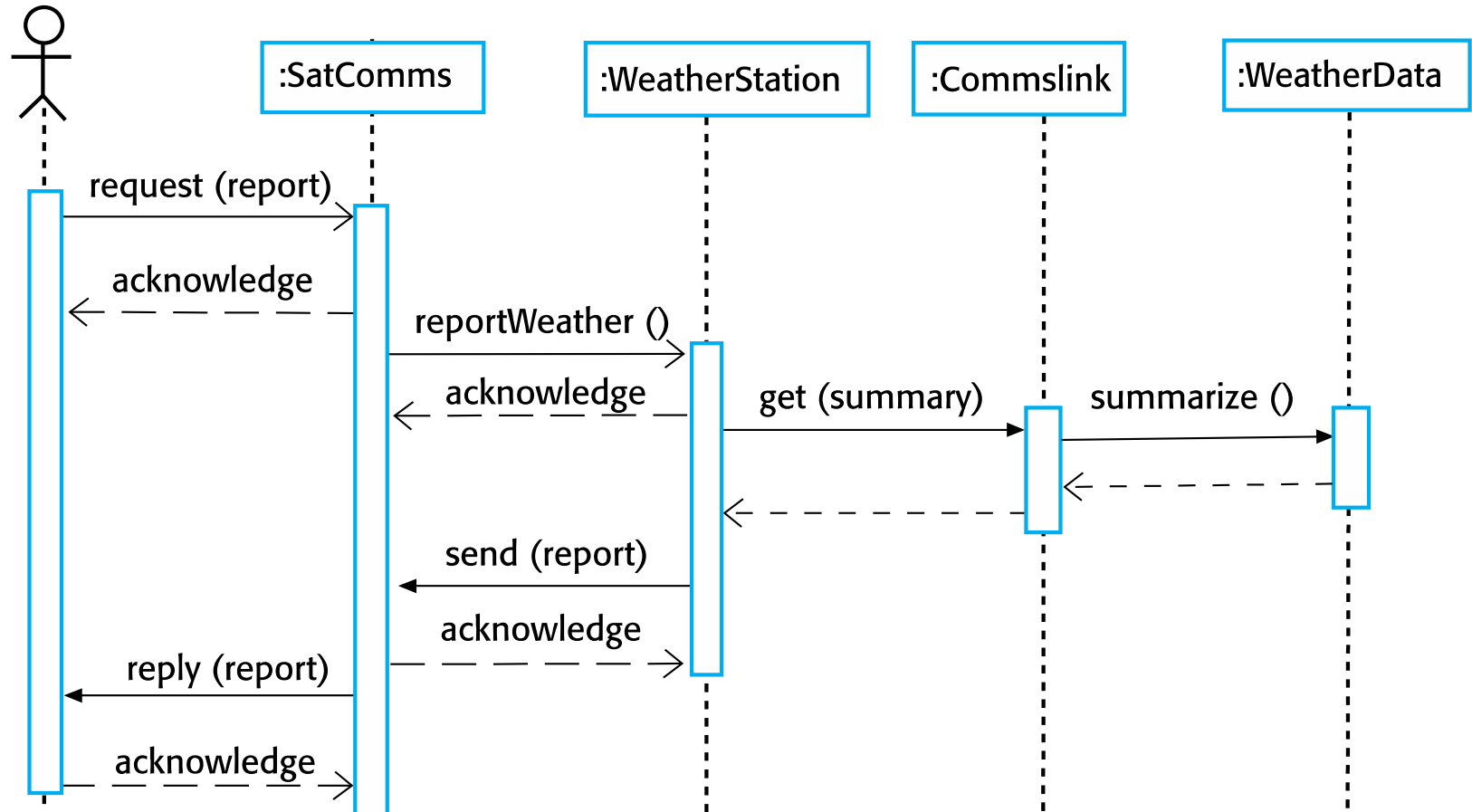


- ✧ **Sequence models** show the sequence of object interactions that take place
 - Objects are arranged horizontally across the top;
 - Time is represented vertically, so models are read top to bottom;
 - Interactions are represented by labeled arrows. Different arrow styles represent different types of interaction.
 - A thin rectangle in an object's **lifeline** represents the time when the object is the controlling object in the system.

Sequence diagram describing data collection



information system

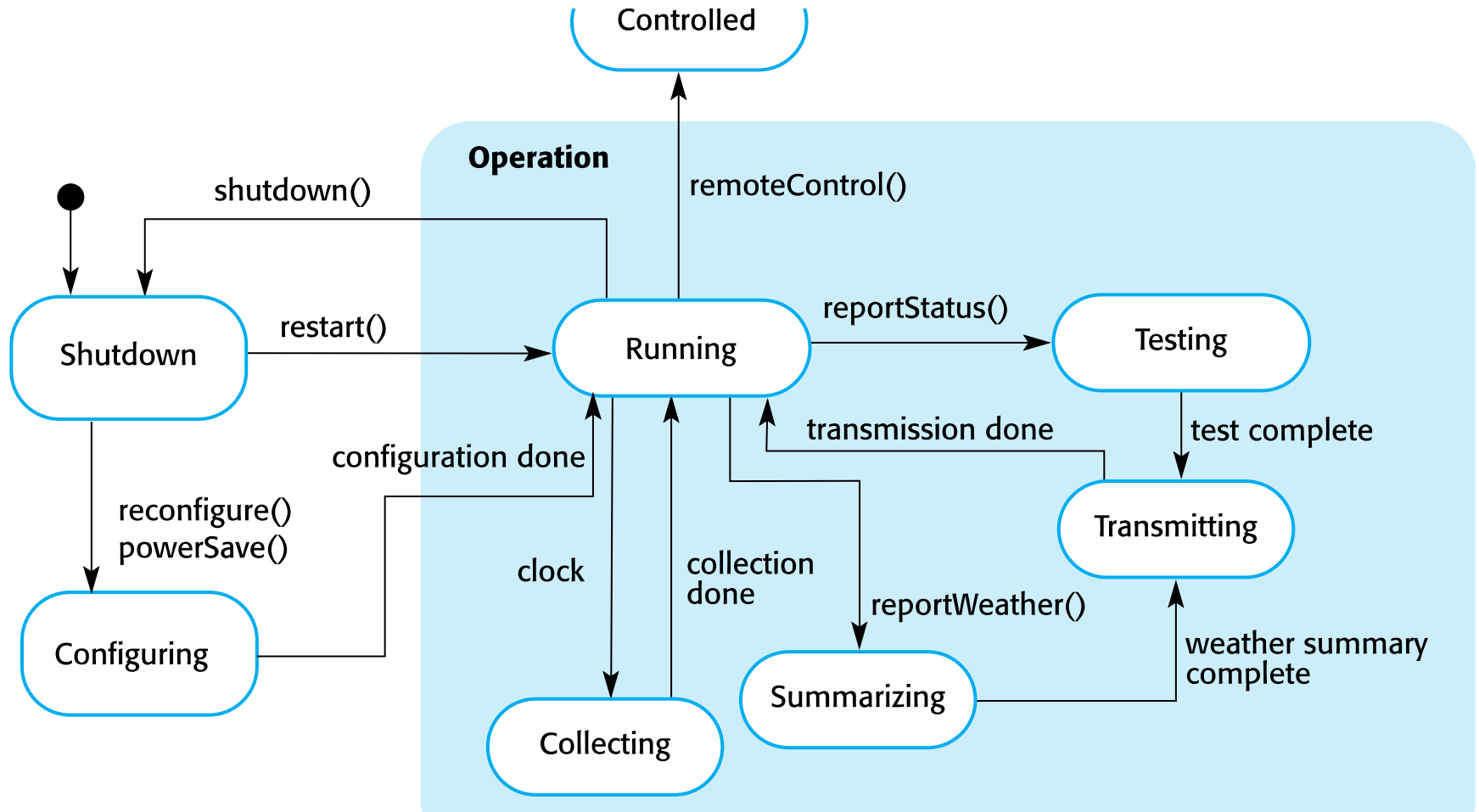


State diagrams



- ✧ **State diagrams** are used to show how **objects respond to different service requests and the state transitions triggered** by these requests.
- ✧ State diagrams are **useful high-level models** of a system.
 - You don't usually need a state diagram for all of the objects in the system.
 - Many of the objects in a system are relatively simple, and a state model adds unnecessary detail to the design.

Weather station state diagram



Interface specification



- ✧ **Object interfaces** - objects and other components can be designed in parallel.
 - Objects may have several interfaces
 - The UML uses class diagrams for interface specification

Weather station interfaces



«interface» Reporting
weatherReport (WS-Ident): Wreport statusReport (WS-Ident): Sreport

«interface» Remote Control
startInstrument(instrument): iStatus stopInstrument (instrument): iStatus collectData (instrument): iStatus provideData (instrument): string



Implementation issues

Implementation issues



✧ Implementation issues that are often not covered in programming texts:

- **Reuse** - Most modern software is constructed by reusing existing components or systems.
- **Configuration management** - Keep track of the many different versions of each software component.
- **Host-target development** **Production software** – a development on one computer (the host system) and execute it on a separate computer (the target system).

Reuse



- ✧ From the 1960s to the 1990s, **most new software was developed from scratch.**
 - The only significant aspect is the reuse of **functions in programming language libraries.**
- ✧ Later, the **reuse of existing software emerged.**
- ✧ Generally **used for business and scientific software.**

Reuse levels



✧ The abstraction level

- You don't reuse software directly, but the abstraction – architecture and patterns

✧ The object level

- You directly reuse objects from a library rather than writing the code yourself.

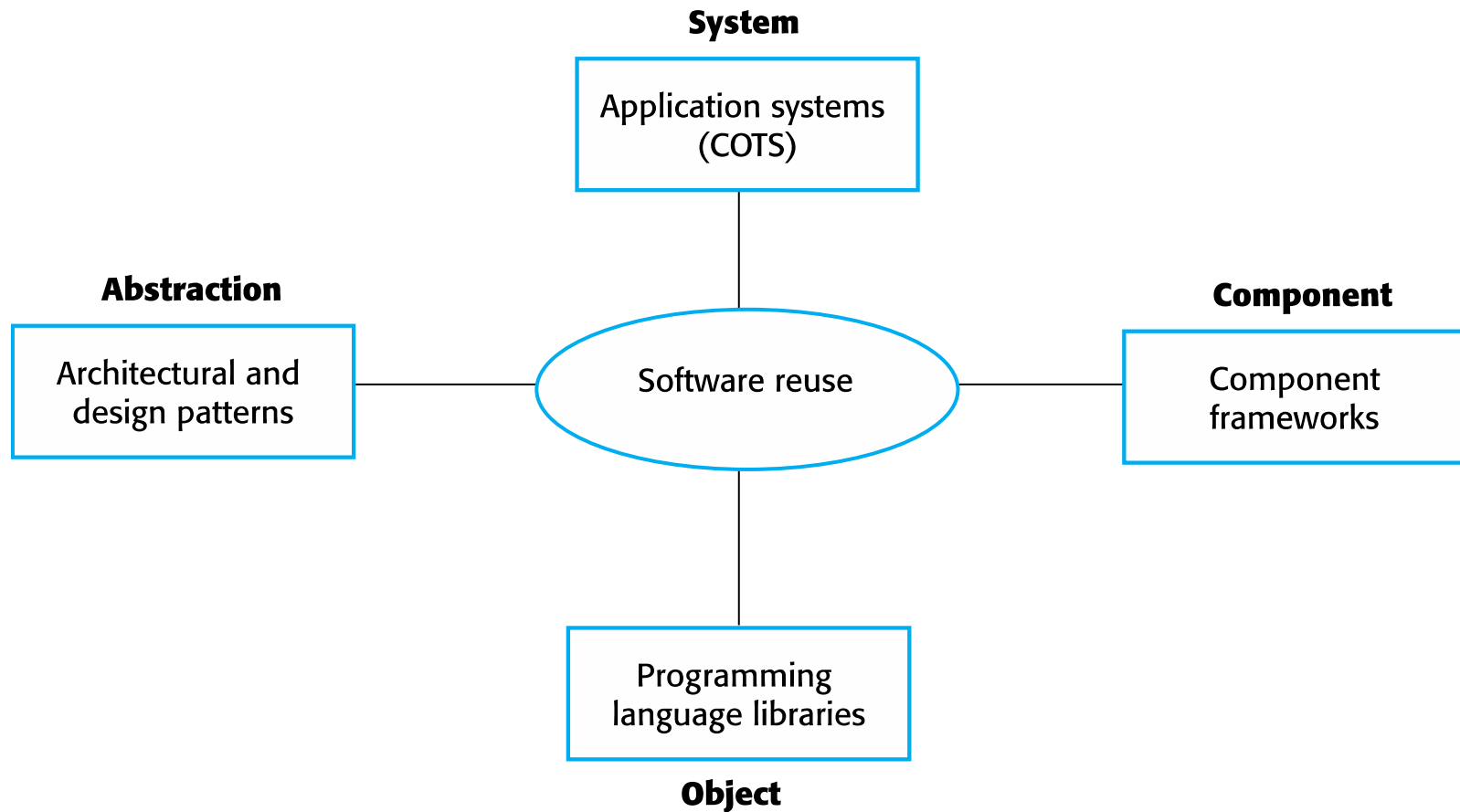
✧ The component level

- Collections of objects and object classes that you reuse in application systems.

✧ The system level

- Reuse entire application systems.

Software reuse





✧ Discuss the following:

- The cost of software reuse vs. building software from scratch
- Proprietary software vs Open-source issues